

Don't see what you're looking for? Log in for the full Help Center experience

 Fireblocks Help Center

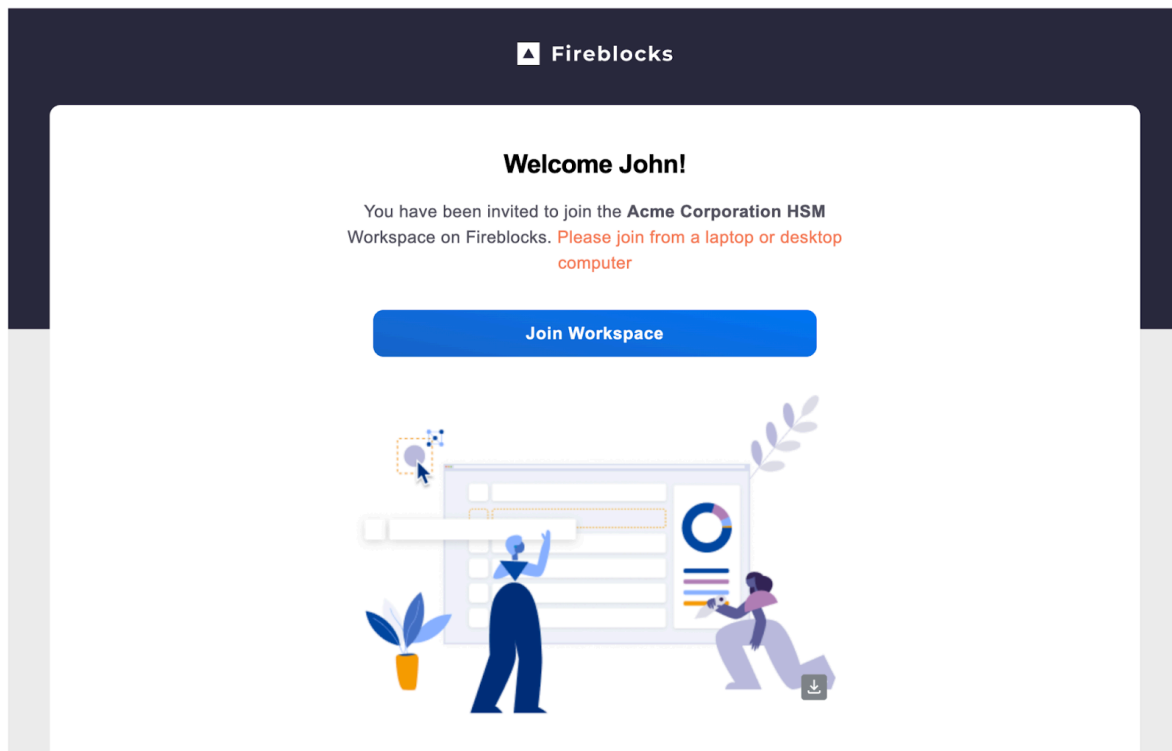
Search the Help Center 

Getting started with Fireblocks Key Link

Updated 4 months ago

Onboarding

When your Fireblocks workspace is created, the workspace Owner receives an email to begin onboarding. Select the link inside the email and follow the steps to join the workspace with the Fireblocks Mobile app (the minimum supported version is **2.7.0**).



Set up the Fireblocks Agent

Once you have successfully onboarded to your workspace, you can set up the Fireblocks Agent that connects to your workspace.

Don't see what you're looking for? Log in for the full Help Center experience

 Fireblocks Help C

/

1. [Install the Fireblocks Agent](#) on the appropriate machine.
2. Run the agent and ensure it's connected to either the example server in the same repository or your actual server. The agent will prompt you to enter a pairing token obtained in the next steps.
3. [Create an API user](#) from your Console with the Signer role.
4. After the Admin Quorum approves the API user, retrieve its pairing token from the Fireblocks Console.
5. Enter the pairing token in the Console where the Fireblocks Agent prompted you to pair the API user with the agent.

Re-enrolling an API user

In your Fireblocks Console, follow these steps:

1. Go to **Settings > Users tab**.
2. Find the API user you want to re-enroll, select **More Actions (...) > Re-enroll API user**.

A request is submitted to the workspace Owner to approve re-enrolling the API user. After the Owner approves the request, you will get a new pairing token for the user in the Console. Use the pairing token you retrieved and enter it in the Console where the Fireblocks Agent prompted to pair the API user with the agent.

Add your keys

You can add and manage keys using the Key Management dashboard in the Fireblocks Console or the [Fireblocks API](#).

Adding keys via the dashboard

The dashboard provides a user-friendly interface for adding validation and signing keys, monitoring their status, and assigning keys to vault accounts.

To add the keys via the dashboard, follow these steps:

1. Navigate to **Settings > External Keys**.
2. Follow the step-by-step prompts to add validation keys and signing keys.

Don't see what you're looking for? Log in for the full Help Center experience

 Fireblocks Help Center

/

To add keys via the [Fireblocks API](#), follow these steps:

1. [Create an API user](#) with the Editor or Admin role.
2. Use the [Fireblocks API](#) or one of the SDKs ([Python](#), [JS](#), [Java](#)) to add your keys.
3. [Configure your Policies](#) to set a designated signer.

Two types of organization-managed keys are used in your Key Link-enabled Fireblocks workspace: validation keys and signing keys.

Validation keys

A validation key is used as proof of authority for the approvals when adding new signing keys to your workspace. This allows a more automated and seamless way of registering and approving new signing keys without needing to approve them manually. Before adding a signing key, the key must be signed by an active validation key previously registered to your workspace, and Fireblocks must verify the signature's validity.

Before adding a new validation key, you should edit your approval group for registering validation keys. In your Fireblocks Console, go to **Settings > Quorums tab > Security & compliance > Add validation keys**. To learn more about approval groups, see [Approval groups](#).

Security & compliance Assign approvers for requests related to workspace security.		
Request	Owner approval	Approval groups
Whitelist addresses	Owner approval ✕	 Admin quorum 2/2 >
Allow one-time addresses	Owner approval ✓	 Admin quorum 2/2 >
Change the TAP	Owner approval ✓	 Admin quorum 2/2 >
Add validation keys	Owner approval ✕	 Admins 1/2 >
Create key backups	Owner approval ✓	 Admin quorum 2/2 >

Use the [Add validation key](#) API endpoint to add a new validation key.

After adding a new validation key, your approval group quorum must approve it using the Fireblocks mobile app. Upon approval, you can register new signing keys.

Signing keys

Don't see what you're looking for? Log in for the full Help Center experience

 Fireblocks Help Center

/

...can be performed either interactively or non-interactively.

Interactive Proof of Ownership

In the interactive proof of ownership method, the signing key is added without any proof attached to the request. Once added to your workspace, Fireblocks sends a challenge through the Fireblocks Agent. The HSM must sign this challenge for the specific key being added. The Agent then relays the signature back to Fireblocks where it is verified. Once verification is complete, the signing key is enabled.

Note

Using this method, the Proof of Ownership message to be signed is computed as `sha256({tenant_id}-{request_id}-{key_id}).hexdigest()`. To verify the message's integrity, you can reconstruct the raw bytes from the request parameters, compute their hash, and then compare it with the hash included in the request. This ensures that you are signing the correct message, as shown below:

```
tenant_id = msg["tenantId"]
request_id = msg["requestId"]
key_id = msg["keyId"]
expected_data_to_sign = f'{tenant_id}-{request_id}-{key_id}'.encode()
data_to_sign = msg["messagesToSign"][0]["message"]
assert data_to_sign == sha256(expected_data_to_sign).hexdigest()
```

Non-interactive Proof of Ownership

In the non-interactive proof of ownership method, the signing key is added with both a proof of ownership message and a signature attached to the request. These elements are included in the `proofOfOwnership` object within the request body, which contains a message and a signature parameter. The expected message format is:

```
Fireblocks|Proof of Ownership Message|<WorkspaceDisplayName>|<SdkApiKey>
```

This message should be in hexadecimal format and is designed to avoid accidental signatures. The components of the message are as follows:

Don't see what you're looking for? Log in for the full Help Center experience

 Fireblocks Help Center

/

- `signingDeviceKeyId`: The ID or label of the signing key in the HSM.
- `UnixTimeInSeconds`: The timestamp reflecting the time of signature creation.

After creating the message and hashing it with SHA256, the HSM should sign the hash. Both the message (in hexadecimal format) and the signature must be included in the request body when adding a signing key. Signing keys added with the `proofOfOwnership` object will not receive an interactive proof of ownership challenge. If the signature is valid, the signing key will be enabled immediately.

Once you decide which proof of ownership method you will use, create and add your signing keys using the following steps:

1. Create a key pair using one of the supported algorithms in your Key Management Infrastructure and get its public key.
2. Create a signed certificate from this public key using an active validation key. The script example below creates certificates that include a public signing key signed by a validation key.

```
#!/usr/bin/env python3
import argparse
# Declare signing key wrapper function
from datetime import datetime, timedelta, timezone

from cryptography import x509
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import ed25519
from cryptography.hazmat.primitives.serialization import (load_pem_public_key)
from cryptography.x509.oid import NameOID

def issue_certificate(root_ca_private_key, certificate_pubkey, commo
# For Ed25519, only the Ed25519 signature algorithm is used
sign_algo = None if isinstance(root_ca_private_key, ed25519.Ed25519P

now = datetime.now(timezone.utc)
time_delta = timedelta(minutes=5)
certificate = x509.CertificateBuilder().subject_name(
```

Don't see what you're looking for? Log in for the full Help Center experience

 Fireblocks Help Center

/

```

x509.NameAttribute(NameOID.COMMON_NAME, issuer_name),
])
).public_key(
certificate_pubkey
).serial_number(
x509.random_serial_number()
).not_valid_before(
now
).not_valid_after(
now + time_delta
).add_extension(
x509.AuthorityKeyIdentifier.from_issuer_public_key(
root_ca_private_key.public_key()
),
critical=False
).sign(root_ca_private_key, sign_algo)

return certificate

def wrap_external_key(validator_privkey, external_public):
COMMON_NAME = "ExampleName"
ISSUER_NAME = "ExampleIssuerName"
cert = issue_certificate(validator_privkey, external_public, COMMON_NAME)
return cert.public_bytes(encoding=serialization.Encoding.PEM).decode()

# Wrap signing key with validation key
encoded_sign_pub_key = load_pem_public_key(sign_pub_key.encode())
signed_key_cert_pem = wrap_external_key(val_priv, encoded_sign_pub_key)
print(signed_key_cert_pem)

```

3. Once you have the signed certificate, upload it to Fireblocks via the [Create signing key](#) endpoint. Note that when you add a new signing key, you must specify the ID of the relevant Fireblocks Agent user you created. This ensures that the relevant Fireblocks Agent receives the signing requests for this key.
4. You can check its status through the [Get signing key by keyId](#) endpoint.
5. Once the certificate is verified, your key will be successfully registered to your workspace. Note that at this stage your signing keys will be set to `enabled=false`

Don't see what you're looking for? Log in for the full Help Center experience

Fireblocks Fireblocks Help Center

- a. Fireblocks will then send a Proof of Ownership signing request to your Fireblocks Agent, which should be signed by your HSM.

b. Once the message is signed, sent back, and verified by Fireblocks, your signing keys will be set as enabled.
7. If you have chosen the non-interactive proof of ownership method, and the message and signature you attached to the request are valid, the key will be enabled.
8. You can now assign your key to vault accounts and create wallets within.

Once you have successfully registered a signing key, you can continue to the next steps: configuring a Policy rule and setting up your Vault. Before you continue, you can add as many keys as you need to your workspace.

Configure a Policy rule

To configure the Policy rule, you must first create an API user with a Signer role for the Fireblocks agent as described [here](#). You can create many different Policy rules for your workspace needs and learn more about them in this [article](#). For each Policy rule, you must ensure the designated signer is set to the API user you created for the Fireblocks Agent.

Learn how to [Set up your Fireblocks Vault with Key Link](#).

Designated signer

Single user

FB Agent User

Was this article helpful?

Don't see what you're looking for? Log in for the full Help Center experience

 Fireblocks Help Center

/

Onboarding

Set up the Fireblocks Agent

Re-enrolling an API user

Add your keys

Configure a Policy rule

Related Articles

-  Token Lifecycle Management Prerequisites
-  Allowlist IP addresses for API user requests
-  Building Retail Service Prerequisites
-  Fireblocks Key Link Overview
-  API user stuck in Pending Setup status
-  Re-enrolling API users
-  Rename and delete API users
-  How to perform Key Backup and Recovery
-  Fireblocks Onboarding Guide
-  Welcome to your Fireblocks journey!
-  Workspace overview and configuration Webinar
-  Add API users
-  Developer Center
-  Workspace Setup and Configuration Videos
-  Treasury Management Prerequisites
-  Clearing and Settlement Services Prerequisites

Don't see what you're looking for? Log in for the full Help Center experience



Fireblocks Help Center



Fireblocks © 2026. All Rights Reserved. NMLS Registration Number: 2066055

[Privacy Policy](#)